

UNIVERSIDADE FEDERAL DE ITAJUBÁ - UNIFEI
CI DIGITAL - PROGRAMA DE DESENVOLVIMENTO DE COMPETÊNCIAS EM
SISTEMAS DIGITAIS

RELATÓRIO TÉCNICO
IMPLEMENTAÇÃO DE MICROARQUITETURA RISC-V PIPELINE

Álvaro Alvarenga Louro

Isabele de Faria Barros

Marcos Cláudio Donato Silva

Pedro Coelho Tagliaferro

ITAJUBÁ, MG

2026

1 INTRODUÇÃO

1.1 Objetivo

O objetivo deste projeto é implementar e validar, em Verilog, uma microarquitetura RISC-V em pipeline de cinco estágios, expandindo os conceitos fundamentais apresentados por Harris e Harris (2019). A arquitetura proposta evolui o modelo de ciclo único para uma estrutura *pipelined* (IF, ID, EX, MEM, WB), integrando módulos avançados como a Unidade de Ponto Flutuante (FPU), para suporte a instruções da extensão RV32F, e a Unidade de Adiantamento (*Forwarding Unit*), responsável pela resolução dinâmica de conflitos de dados. A validação funcional é realizada por meio de *testbenches* e simulações baseadas em waveforms. Adicionalmente, o projeto contempla o fluxo de síntese lógica utilizando a ferramenta Cadence Genus, possibilitando a análise crítica de métricas de PPA (Potência, Performance e Área) e a verificação de restrições de *timing* (STA) sob diferentes cenários de frequência.

1.2 Arquitetura RISC-V e Modelo Pipelined

A arquitetura RISC-V é um conjunto de instruções (ISA) baseado no modelo RISC (*Reduced Instruction Set Computer*), projetada para ser aberta, modular, extensível e de uso livre. Sua simplicidade e padronização tornam-na adequada tanto para aplicações acadêmicas quanto industriais, favorecendo a construção de processadores didáticos e sistemas embarcados de alto desempenho.

A arquitetura define formatos de instrução bem estruturados — como os tipos R, I, S, B e J — e um conjunto de 32 registradores de propósito geral (x0 a x31). Para atender aos requisitos deste projeto, a implementação suporta também a extensão RV32F, incorporando registradores específicos para ponto flutuante e instruções aritméticas dedicadas. Essas características facilitam a implementação modular da microarquitetura proposta.

Diferente da abordagem de ciclo único, este projeto implementa um processador em pipeline de 5 estágios (IF, ID, EX, MEM, WB). Essa técnica aumenta o *throughput* (vazão) de instruções ao permitir que múltiplas etapas de diferentes instruções sejam processadas simultaneamente. Para mitigar os conflitos de dados (*hazards*) inerentes ao paralelismo, foram integradas uma Unidade de Adiantamento (*Forwarding Unit*) e lógicas de controle de fluxo. A implementação segue a base teórica de Harris e Harris, expandida com os conceitos de *timing* e análise de PPA (Potência, Performance e Área) exigidos para a síntese física.

2 ARQUITETURA

2.1 Base Teórica

Como mencionado anteriormente, a base teórica deste projeto expande os fundamentos do livro *Digital Design and Computer Architecture – RISC-V Edition* (Harris & Harris, 2019), evoluindo a arquitetura de ciclo único para um modelo de Pipeline de 5 estágios, conforme abordado no módulo SD212. Esta implementação exige uma reestruturação do *datapath* para suportar a execução paralela de instruções, além da adição de lógica complexa, como a Unidade de Adiantamento (*Forwarding Unit*) para resolução de *hazards* de dados e a integração de uma FPU (*Floating Point Unit*).

Além da microarquitetura, o projeto se fundamenta em conceitos multidisciplinares para garantir a validade física e funcional do circuito: a etapa de síntese lógica utiliza scripts TCL para a ferramenta Cadence Genus, baseada no módulo SD221; a análise de desempenho foca em Static Timing Analysis (STA) e métricas de PPA (*Power, Performance, Area*), vistas em SD232.

2.2 Diagrama Geral do *Datapath*

A imagem a seguir apresenta o diagrama de blocos expandido da arquitetura, projetado para suportar a execução de instruções via Pipelining e operações matemáticas complexas. Diferentemente do modelo monociclo básico, esta estrutura integra a Unidade de Ponto Flutuante, que opera em paralelo com a ALU principal para processar números reais, e a Unidade de Adiantamento, responsável por mitigar conflitos de dados (*data hazards*) sem a necessidade de paralisar o fluxo de execução.

O diagrama evidencia a segmentação do *datapath* em estágios distintos e as novas linhas de controle necessárias para gerenciar a dependência de dados entre instruções consecutivas. A coordenação entre esses módulos visa maximizar a vazão do processador, garantindo que múltiplas instruções sejam processadas simultaneamente de forma eficiente.

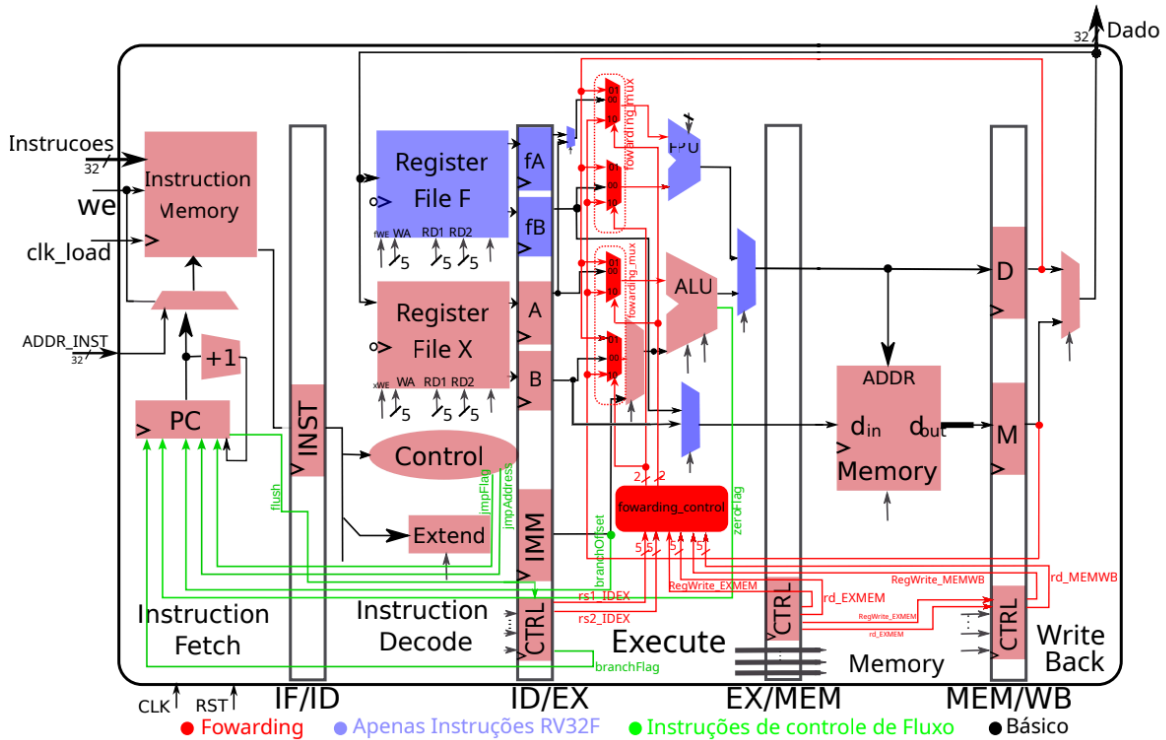


Figura 1. Arquitetura de um processador RISC-V pipeline.

2.3 Implementações em Verilog dos Componentes Principais da Arquitetura

Nesta seção, serão descritos os principais elementos constituintes da microarquitetura implementada em Verilog.

2.3.1 Memórias

Os módulos de memória utilizados neste projeto foram implementados de modo a atender às necessidades do pipeline RISC-V. A Memória de Instrução é responsável por fornecer instruções de 32 *bits* ao processador, sendo acessada de forma assíncrona durante o estágio de *Instruction Fetch*. Já a Memória de Dados é *byte-addressable* e suporta operações de leitura assíncrona e escrita síncrona, sendo utilizada pelas instruções de carregamento e armazenamento. A seguir, são apresentados os detalhes de cada um desses módulos.

2.3.1.1 Memória de Instrução – *instruction_memory.v*

Memória que armazena todas as instruções do *software*, que são carregadas previamente e acessadas sequencialmente ou por desvios. Sua função primordial durante a execução é fornecer a instrução correta ao processador para decodificação. Este módulo possui uma entrada de endereço (A), pela qual recebe o valor do *Program Counter* (PC) indicando o endereço que contém a instrução desejada. Como saída, ela gera a instrução (Instr, na saída RD) completa de 32 *bits* que está armazenada no endereço especificado. Em uma arquitetura de ciclo único, a Memória de Instrução opera de forma combinacional e é acessada no estágio inicial de busca de instrução.

2.3.1.2 Banco de Registradores X e F - *register_file.v*

Memória que contém registradores de propósito geral, guardando valores e resultados de operações de forma temporária. São manipulados de forma direta pelas instruções. Possui três entradas de endereço (A1 que indica o endereço de leitura 1, A2 que indica o endereço de leitura 2 e A3 que indica o endereço de escrita), uma entrada de dados (WD3), uma entrada de permissão de escrita (WE) e duas saídas de leitura de dados (RD1 e RD2). Na arquitetura, existem duas instâncias deste módulo, uma para os valores inteiros e outra para os valores *float*. Válido notar que nesta implementação os registradores estão síncronos na borda de descida.

2.3.1.3 Memória de Dados - *data_memory.v*

A Memória de Dados é um componente essencial do *datapath*, atuando como o local para o armazenamento de dados variáveis durante a execução do programa. Diferente da Memória de Instrução, que é acessada para buscar o código, a Memória de Dados é manipulada por instruções de carregamento (*load*) e armazenamento (*store*). Para sua operação, a Memória de Dados possui uma entrada de endereço (A) que especifica a localização de memória a ser acessada, uma entrada de dados para escrita (WD) que recebe o valor a ser armazenado, uma entrada de controle (WE - *Write Enable*) que habilita a operação de escrita, e uma saída de dados lidos (RD - *ReadData*) que fornece o valor recuperado do endereço especificado. Essa memória é crucial para as operações que envolvem manipulação de variáveis e estruturas de dados dinâmicas do programa.

Por ser acessível por *byte*, esta memória contém quatro módulos menores de memória de um *byte*, que, quando lidas em sequência, formam a palavra de 32 *bits*. Um módulo *decoder* que gerencia qual dessas memórias vão ser acessadas para a escrita, um *manager* é responsável pelo gerenciamento da leitura.

2.3.3 ALU (Unidade Lógica e Aritmética) - *ALU.v*

A Unidade Lógico-Aritmética (ULA) executa diversas operações entre os operandos de entrada A e B, de 32 *bits*, controladas por uma entrada de seleção *ALUControl* de 3 *bits*. As operações suportadas e seus respectivos códigos são:

- Adição com a entrada de seleção 000;
- Subtração com a entrada de seleção 001;
- Operação lógica *AND* com 010;
- Operação lógica *OR* com 011;
- "*Set Less Than*" (Slt) com a entrada de seleção 101.

A ULA integra ainda uma saída de *flag* 'Zero', utilizada para indicar o resultado de operações e habilitar desvios condicionais.

2.3.4 FPU (Unidade de Ponto Flutuante) - *FPU.v*

A unidade de ponto flutuante executa diversas operações com números de ponto flutuante de 32 bits no padrão IEEE 754 entre as entradas A e B, controladas de forma semelhante a ALU com o sinal *selFPU*. As operações suportadas e seus códigos são:

- Adição com a entrada de seleção 0100;
- Subtração com a seleção 0101;
- Multiplicação com a seleção 0110;
- Mínimo com a seleção 0111;
- Máximo com a seleção 1000;
- Igual com a seleção 1001;
- Menor que com a seleção 1010;
- Menor ou igual com a seleção 1011;
- Mover de Register File X para Register File F com a seleção 1100;
- Mover de Register File F para Register File X com a seleção 1101;
- Converte inteiro para ponto flutuante com a seleção 1110;
- Converte ponto flutuante para inteiro com seleção 1111.

2.3.3 Unidade de controle

A Unidade de Controle é composta por três outros módulos menores, o *Main Decoder* (Decodificador Principal), o *ALU Decoder* (Decodificador da ULA) e *FPU Decoder*. Sua função é interpretar a instrução atual e gerar os sinais de controle necessários para orquestrar as operações dos componentes do *datapath*, garantindo a execução correta da instrução.

2.3.3.1 Decodificador Principal - *Main_Decoder.v*

O *Main Decoder* analisa o campo Opcode (*bits* 6:0) da instrução RISC-V e a partir disso ele gera sinais de controle que direcionam o fluxo de dados e as operações.

2.3.3.2 Decodificador da ULA - *ULA_Decoder.v*

O ALU Decoder, por sua vez, é responsável por gerar o sinal ALUControl de 3 *bits*, que especifica a operação exata que a Unidade Lógico-Aritmética (ULA) deve executar. Para isso, ele utiliza informações do campo funct3 (*bits* 14:12), do campo funct7 (bit 30) e, em alguns casos, do próprio Opcode (sinal ALUOp vindo do Main Decoder). Juntos, estes campos permitem que o ALU Decoder determine a operação a ser realizada pela ULA.

2.3.3.3 Decodificador da FPU - *FPU_Decoder.v*

O decodificador da FPU é responsável por ordenar a operação realizada pela FPU, ele recebe os campos funct5 e rm da instrução e, a partir disso, atua no sinal de seleção da unidade de ponto flutuante.

2.3.4 MUXes-

Os Multiplexadores são componentes essenciais na arquitetura do *datapath*, atuando como seletores de dados. Sua função principal é escolher uma entre várias entradas de dados e rotear a entrada selecionada para uma única saída. Essa seleção é determinada por um conjunto de sinais de controle específicos, fornecidos pela Unidade de Controle.

MUX do PC (Estágio Fetch): Seleciona o próximo valor do Contador de Programa, alternando entre a execução sequencial ($PC + 4$) e o endereço alvo de um desvio ou salto (*Branch/Jump*).

MUX do Operando B (Estágio Execute): Define a segunda entrada da ULA, selecionando entre o valor lido do Registrador *rs2* e o valor Imediato estendido (utilizado em instruções tipo-I e *Stores*).

MUXes de Forwarding (Estágio Execute - Destaque em Vermelho): Conjunto crítico de seletores (para entradas A e B da ULA e FPU) que escolhem a origem do operando: o dado vindo do estágio de Decodificação (fluxo normal) ou dados adiantados diretamente dos estágios de Memória ou *Write-Back* para resolver *hazards* de dados.

MUX de Write-Back (Estágio Write-Back): Seleciona o dado final a ser escrito no banco de registradores, escolhendo entre o resultado processado (pela ULA ou FPU) ou o dado lido da Memória de Dados (*Load*).

2.3.5 Contador de Programa - *PC.v*

O Contador de Programa é um registrador fundamental em qualquer arquitetura de processador. Sua principal função é armazenar o endereço de memória da instrução que está sendo buscada para execução no ciclo de *clock* atual. Ele atua como um ponteiro que aponta para a próxima instrução a ser processada pelo sistema. Além disso, é o componente chave para determinar a sequência de execução das instruções, seja de forma sequencial ou através de desvios e saltos.

Em um processador de ciclo único, o PC é atualizado a cada ciclo de *clock*. A sua próxima instrução (*PCNext*) pode ser determinada por dois caminhos principais:

- Execução Sequencial: Na maioria dos casos, o PC é incrementado em 4 (*PCPlus4*), pois cada instrução RISC-V tem 4 bytes de comprimento. Isso garante que o processador busque a próxima instrução na sequência do programa.
- Desvios e Saltos: Para instruções de desvio (*branches*) ou salto (*jumps*), o PC é carregado com um novo endereço de destino (*PCTarget*), que é o resultado de um cálculo de endereço (soma do PC atual com um imediato estendido, por exemplo) ou de um registrador. A escolha entre *PCPlus4* e *PCTarget* é feita por um multiplexador controlado pelo sinal *PCSrc*, gerado pela Unidade de Controle.

Por ser um registrador, o PC é um elemento síncrono, ou seja, seu valor é atualizado apenas na borda de subida do sinal de *clock*, garantindo uma transição controlada entre os endereços das instruções.

2.3.6 Extensor de Sinal - *Sign_Extend.v*

O módulo Extensor de Sinal tem a função de converter os valores imediatos (constantes) de diferentes tamanhos presentes nas instruções para 32 *bits*. Ele realiza isso através de extensão de sinal para valores com sinal ou extensão de zero para valores sem sinal, garantindo que o valor imediato (*ImmExt*) esteja no formato correto para uso pela ULA ou em cálculos de endereços, sob controle do sinal *ImmSrc*. A tabela 3 mostra as codificações para *ImmSrc* para as funções implementadas.

2.3.7 Forwarding Control Unit - *forwarding_control.v*

O módulo centraliza a resolução de conflitos de dados e controle do pipeline, unificando as lógicas de adiantamento e pausa. O circuito implementa o *forwarding* ao monitorar os operandos no estágio de execução e, caso detecte conflitos com instruções nos estágios de Memória ou Escrita, adianta o dado mais recente (priorizando a Memória) para

evitar paradas desnecessárias. Paralelamente, o módulo gerencia a inserção de *Stalls* (congelamento do PC e estágios iniciais) e *Flushes* (limpeza de registradores) para lidar com situações críticas onde o adiantamento não é suficiente, especificamente em casos de dependência de *Load-Use*, operações aritméticas complexas (multiplicação) e latência variável da unidade de divisão, além do descarte de instruções após desvios de fluxo.

2.3.8 Arquitetura Completa (Módulo Topo) - topo.v

O processador apresentado baseia-se na arquitetura RISC-V e é estruturado em um pipeline de 4 estágios *Instruction Fetch*, *Instruction Decode*, *Execute/Memory* e *Write Back*.

Uma característica distintiva desta topologia é a fusão das etapas de processamento e acesso a dados em um único estágio (*Execute/Memory*). Nesta configuração, não há interposição de registradores (*flip-flops*) entre a saída da Unidade Lógica e Aritmética (ULA) e o bloco de Memória. Consequentemente, o cálculo de endereços ou dados e o eventual acesso à memória ocorrem sequencialmente dentro do mesmo ciclo de clock.

Embora reduza a latência total de estágios, essa arquitetura está sujeita a Data Hazards (conflitos de dados). Ocorre um hazard quando uma instrução subsequente necessita de um operador que ainda está sendo processado no estágio de Execução/Memória e ainda não foi consolidado no banco de registradores (estágio de *Write Back*).

3 SIMULAÇÃO E VALIDAÇÃO

3.1 Metodologia de Testes

Para a validação do projeto, a metodologia adotada consistiu na criação de *testbenches* para cada módulo, permitindo a verificação individual do comportamento de cada componente. As simulações foram realizadas utilizando a ferramenta *Xcelium*, com a análise dos sinais e o comportamento esperado sendo observados através de *waveforms*. Após a validação individual, os módulos foram integrados no módulo de nível superior (topo.v) com descrição estrutural, onde um novo *testbench* verificou a funcionalidade do processador.

3.2 Resultados

O teste teve como foco garantir o comportamento esperado do processador, para tal foi realizado teste conforme estrutura proposta no documento “Roteiro de atividades” em

Etapa 2: Testes e Simulação com o seguinte código proposto para o teste:

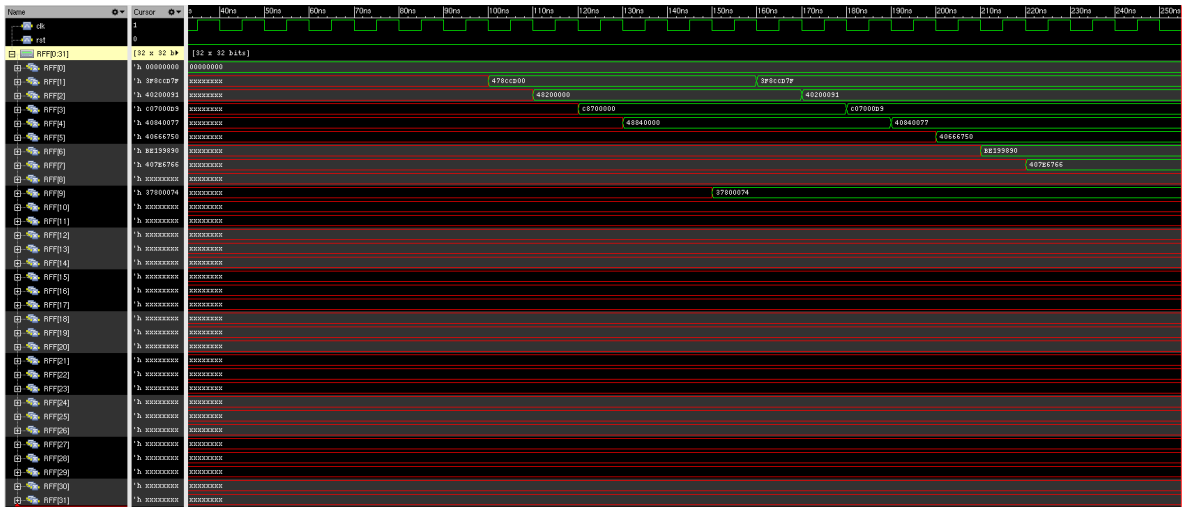
```

1  .data
2  val1:.word72090  #1.1 em Q16.16
3  val2:.word163840 #2.5
4  val3:.word-245760 #-3.75
5  val4:.word270336 #4.125
6  result_fp:.word0  #espaço para FP
7  result_int:.word0 #espaço para inteiro
8  .text
9  .globl _start
10 _start:
11  #----carregar valores inteiros Q16.16----
12  lw x5,val1
13  lw x6,val2
14  lw x7,val3
15  lw x8,val4
16  #----converter direto para ponto flutuante----
17  fcvt.s.wf1,x5
18  fcvt.s.wf2,x6
19  fcvt.s.wf3,x7
20  fcvt.s.wf4,x8
21  #----multiplicar cada valor por 2^-16 para corrigir Q16.16
22  #----
23  li x10,0x37800074 #2^-16=1.52587890625e-05 em IEEE754
24  fmv.w.xf9,x10 #f9 = constante 2^-16
25  fmul.sf1,f1,f9
26  fmul.sf2,f2,f9
27  fmul.sf3,f3,f9
28  fmul.sf4,f4,f9
29  #----somas em ponto flutuante ----
30  fadd.sf5,f1,f2
31  fadd.sf6,f5,f3
32  fadd.sf7,f6,f4 #resultado final FP
33  #----salvar resultado FP em memória----
34  fswf7,result_fp,0
35  #----converter resultado FP -> inteiro e salvar ----
36  fcvt.w.sx9,f7 #converte float para inteiro (trunca)
37  sw x9,result_int,0
38  end:
39  beq x0,x0,end #loop infinito

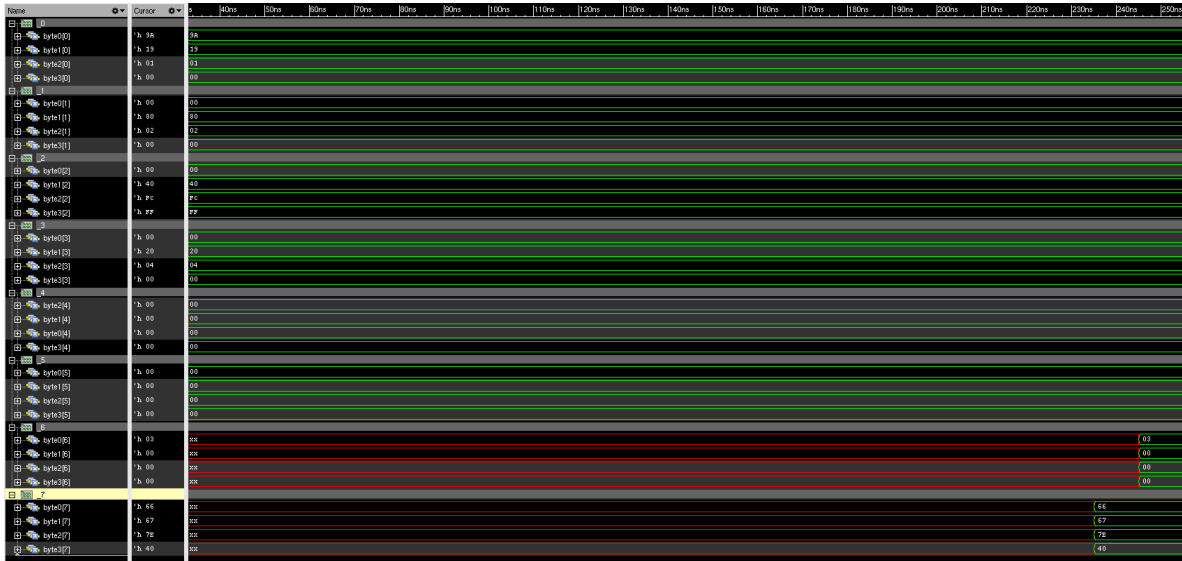
```

Registadores:





Memória:



4 SÍNTESE E RESULTADOS

4.1 Definição dos Cenários

A síntese foi executada para três restrições temporais distintas. A Tabela abaixo resume os principais resultados obtidos pelo *report* de *qor* (*Quality of Results*):

	Baseline (30ns)	PPA1 (20ns)	PPA2 (10ns)
Slack (Timing)	+10,246ns	+3,143ns	+0,007ns
Potencia	0,252 mW	0,378 mW	0,867 mW
Área	57119,198 μm^2	57123,576 μm^2	57744,648 μm^2

Tabela 1: Resultados da síntese em três cenários diferentes.

4.2 Análise dos caminhos críticos

4.2.1 Baseline

No caso base, o caminho crítico se mostrou no estágio ID, mais precisamente, o caminho consiste em buffers e, por fim, o ponto final é um registrador do banco de registradores F.

4.2.2 PPA1

O caminho crítico foi identificado no caso de flush, o lançamento ocorreu no flip-flop de entrada do estágio EX/MEM, seguido por um atraso combinacional e posteriormente, o atraso de soma da ALU. Em seguida, após mais um atraso combinacional, retorna ao estágio IF, no módulo de contador de programa, isso indica que a instrução executada foi a instrução beq.

4.2.3 PPA2

O caminho crítico se mostrou no estágio EX/MEM, especificamente na operação de soma da FPU. O lançamento ocorreu no flip-flop de entrada do estágio EX/MEM, existe então um atraso da lógica de controle da FPU, seguido do processo de soma com ponto

flutuante (decomposição, alinhamento, soma, normalização e composição), para então chegar à saída da FPU e então ocorrer a captura no flip-flop de entrada do estágio WB.

5 ANÁLISE DE PPA

A análise comparativa demonstra viabilidade do circuito a 100MHz (PPA2), contudo, sua margem é muito pequena (0,007ns) para uma implementação funcional, indicando o limite da implementação atual. Observa-se que, em frequências moderadas (PPA1) o caminho crítico imposto é relacionado ao controle de fluxo (tratamento de *branch hazard*), para este cenário, a instrução *beq* é o que gera maior atraso.

Em frequências mais altas o gargalo principal reside na latência da operação de soma da Unidade de Ponto Flutuante, sugerindo que para frequências superiores a 100MHz, seria necessário separar os estágios de execução e memória ou implementar um pipeline na FPU (FPU multi-estágio).

O gráfico abaixo apresenta a relação entre frequência, potência e slack:

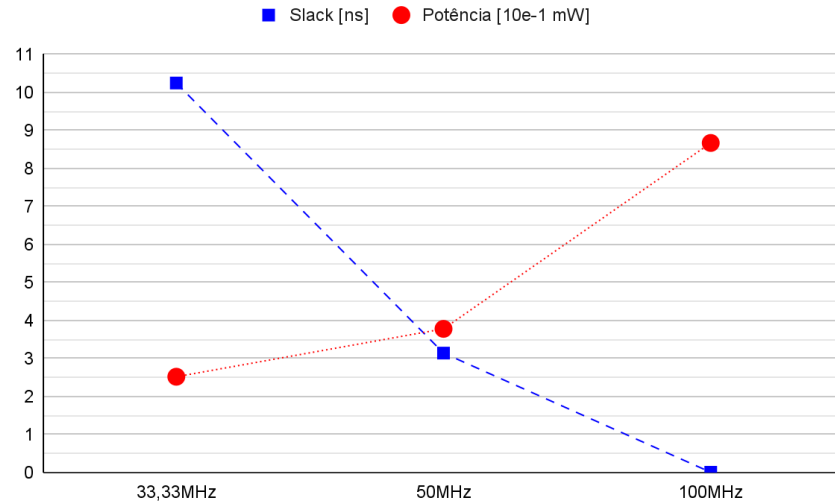


Gráfico 1: Frequência x slack x potência.

O gráfico a seguir mostra a relação entre potência e frequência:

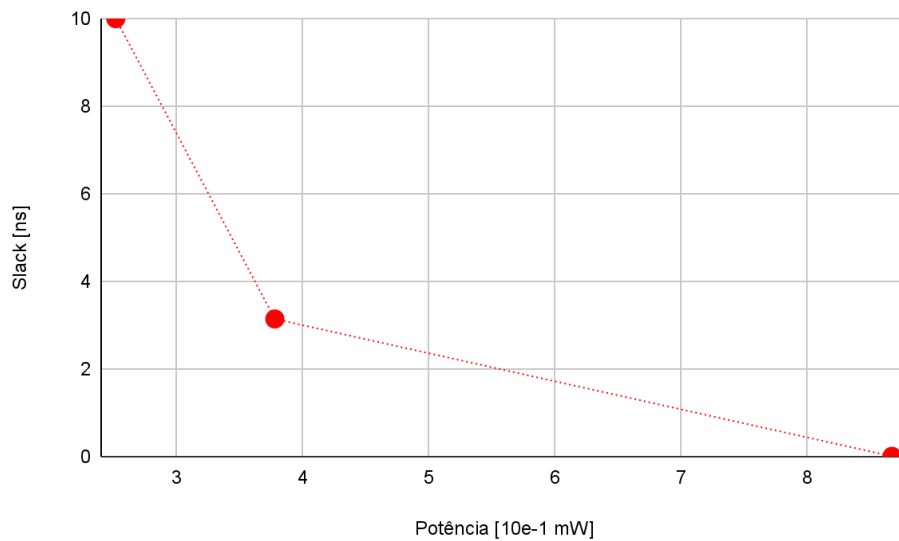


Gráfico 2: Slack x potencia.

Essas relações nos indicam que existe o aumento de potência e diminuição do slack em frequências maiores. O aumento da frequência também aumenta drasticamente a potência utilizada pelo circuito, uma variação de 50% no período (PPA1 para PPA2) resultou em um aumento de 229,36% na potência.

6 CONCLUSÃO

O desenvolvimento do processador demonstrou que a arquitetura implementada é capaz de operar dentro das restrições temporais estabelecidas, tendo slack positivo em todos os cenários. Contudo, o cenário ótimo para a implementação é o que possui período de 20ns no *clock*.

O controle de fluxo é funcional e consegue evitar inconsistências providas da arquitetura em pipeline e o módulo de *forwarding control* realiza o seu papel em diminuir a quantidade de bolhas quando um dado precisa ser utilizado antes de estar disponível em registradores.

Apesar do sucesso da síntese física e funcional, o circuito apresenta limitações, o módulo de *forwarding control* implementado não é capaz de tratar todas as possibilidades de *hazards*. Em cenários específicos, este módulo consegue de realizar o manejo de dados de forma adequada, ainda sendo necessário bolhas e cuidado com a sequência das instruções, endereço de destino e valor imediato em instruções adjacentes.

Um detalhe importante em relação à arquitetura e manejo dos *data hazards* é o fato de que os módulos registradores (rff e rfx) foram alterados para terem sensibilidade ao clock em borda de descida. Isso foi feito para “adiantar” o dado que é requisitado no estágio EX/MEM. Dessa forma, a informação leva meio ciclo a menos para atingir o local desejado.

REFERÊNCIAS

HARRIS, David Money; HARRIS, Sarah L. *Digital Design and Computer Architecture: RISC-V Edition*. Cambridge: Elsevier, 2019.

MSYKSPHINZ. RISC-V ISA Documentation Generator. Disponível em: <https://msyksphinz-self.github.io/riscv-isadoc/>. Acesso em: 30 jul. 2025.